C++ Chapter 3-4 Study Notes: Ordered Version

Variables, arrays, strings, structs, pointers, new/delete, dynamic struct arrays

Special note: this version is not just an appendix at the end. The ideas are reorganized in learning order, while keeping the same “box, row of boxes, related data, address” way of thinking.

0. First, build one main line

The core of Chapter 3 and Chapter 4 is not memorizing syntax. The real goal is to see how data is stored in memory.

```
normal variable -> array -> C-style string -> string -> struct -> pointer -> new/delete ->
dynamic array -> dynamic struct array
```

A normal variable is like one box. An array is like a row of boxes. A struct is like tying several related boxes into one bigger box. A pointer is also a box, but the thing inside it is not an ordinary value. It is an address.

1. Normal variable: one box

```
int a = 1;
```

This variable can be understood as a box. The box has at least four pieces of information:

Concept	Example
Name	a
Type	int
Content	1
Address	for example 0x1000

Usually we only care about the content, for example a contains 1. But once we enter arrays and pointers, the address becomes very important.

2. Array: a row of continuous boxes

```
int sales[12];
```

Why do we need arrays? Suppose we have sales data for a whole year. If every month is written as a separate variable:

```
int jan;

int feb;
```

```
int mar;

int apr;

// ... all the way to dec
```

This works, but it is very inefficient to manage. With an array, the 12 months can be placed in one row:

```
int sales[12];
```

Abstractly, it looks like this:

```
sales[0]  sales[1]  sales[2]  ...  sales[11]
```

They are stored as one continuous area in memory, so the program can quickly find an element by using its index.

Special reminder: sales[12] is not the 12th element. It is outside the array. The legal indexes of sales[12] are 0 through 11. C++ may not check this error for you, so it may produce strange results.

2.1 Array initialization

```
int sales[12] = {22, 33, 13, 5};
```

The first 4 elements are assigned values. The elements not written out are initialized to 0.

```
sales[0] = 22

sales[1] = 33

sales[2] = 13

sales[3] = 5

sales[4] to sales[11] = 0
```

By the way, C++11 supports this cleaner syntax:

```
double earnings[4] {1.2e4, 1.6e4, 1.1e4, 1.7e4};
```

3. C-style string: a special char array

A C-style string is essentially a char array, but it has one very special rule: it must end with the ending marker \0.

```
char name[5] = {'J', 'o', 'h', 'n', '\0'};
```

The \0 tells the program: the string ends here.

Special reminder: a single character uses single quotes, such as 'a'. A string literal uses double quotes, such as "John".

3.1 Why can cin >> name only read one word?

```
char name[20];

cin >> name;
```

If you enter John Lee, the result is usually only John. This is because cin >> reads by whitespace. It stops when it sees a space.

You can imagine the input queue like this:

```
J o h n [space] L e e [newline]
```

cin >> name stops when it sees the space, so it only takes John.

3.2 Read a whole line: cin.get and cin.getline

Syntax	Behavior	Feature
cin >> name	Stops at a space	Only reads one word
cin.get(name, 20)	Reads part of the line, but leaves newline in the queue	You need to handle newline later
cin.get(name, 20).get()	Reads the content first, then consumes newline	More complete
cin.getline(name, 20)	Reads until newline, then replaces newline with \0	Best for beginners when reading a whole line

```
char name[20];

cin.getline(name, 20);
```

3.3 char arrays cannot be assigned as a whole

```
char name1[20] = {};

char name2[20] = "John";
```

```
name1 = name2;  // wrong: arrays cannot be assigned like this
```

If you want to copy a C-style string, include cstring:

```
#include <cstring>


strcpy(name1, name2);
```

strcpy means: copy the right-side string into the left-side char array.

strlen(name) counts the visible characters before \0. sizeof(name) counts how many bytes the whole array occupies.

```
char name[10] = "Tom";


strlen(name)  // 3

sizeof(name)  // 10
```

4. string: more like an object

char arrays are very low-level, so many operations are annoying. C++ string is much more convenient.

```
string name = "John";

string good = "You are so nice";

string result;


result = name + " " + good;
```

The output is:

```
John You are so nice
```

+ is like a joining worker, and += is like appending to the existing result:

```
string name = "John";
```

```
name += " You are so nice";
```

string treats text as an object, so we can directly use methods:

```
string name = "John";

int len = name.size();
```

If you want to read a whole line into a string:

```
string name;

getline(cin, name);
```

Special reminder: int, float, and double are usually read with cin >>. getline is mainly for string or char arrays when reading a whole line. It is not used to directly read int or float.

5. struct: tie related data into a new type

A struct lets the program understand: this group of data belongs together. For example, a car is not only a price. It also has an id, a name, and a price.

```
struct Car {

    int id;

    string name;

    float price;

};
```

This is not creating a variable. This is defining a new type. It means: I am telling C++ what a Car-shaped piece of data should look like.

Creating a variable looks like this:

```
Car bmw;
```

Here, Car is the type, and bmw is a concrete object.

5.1 Use . to access struct members

```
bmw.id = 1;
```

```
bmw.name = "X5";

bmw.price = 68300.0f;
```

When an object accesses its own members, use the dot operator .

```
cout << bmw.id << endl;

cout << bmw.name << endl;

cout << bmw.price << endl;
```

5.2 Initialize a struct at once

```
Car bmw = {

    1,

    "X5",

    68300.00f

};
```

This style must follow the order of the struct members: id, name, price.

5.3 Struct array

```
Car cars[3] = {

    {1, "BMW", 100000},

    {2, "Toyota", 25000},

    {3, "Honda", 28000}

};
```

Here, cars is a group of Car objects. cars[0] is the 0th Car object. cars[0].price is the price member of the 0th Car.

```
cars[0].id

cars[0].name

cars[0].price
```

6. Pointer: a variable that stores an address

This is a very important part of Chapter 4. A pointer is also a variable, but it only manages addresses.

```
int num = 6;
```

This variable can be understood like this:

Concept	Example
Name	num
Type	int
Content	6
Address	for example 0x1000

Now create a pointer variable:

```
int* pointer;

pointer = &num;
```

int* pointer means: pointer is a pointer, and it stores the address of an int variable. &num means the address of num. So pointer = &num means putting num's address into pointer.

6.1 Do not mix pointer, &num, and &pointer

```
cout << num << endl;       // content of num: 6

cout << &num << endl;      // address of num

cout << pointer << endl;   // address stored inside pointer, which is num's address

cout << *pointer << endl;  // content inside the address pointer points to: 6

cout << &pointer << endl;  // address of the pointer variable itself
```

Expression	Meaning	Possible output
num	Content of the normal variable	6
&num	Address of the num box	0x1000
pointer	The address stored inside pointer	0x1000
*pointer	Go to the address stored in pointer and get the content	6
&pointer	The address of the pointer box itself	for example 0x2000

Special reminder: pointer and &num are usually the same, because pointer stores num's address. But &pointer is not num's address. It is the address of the pointer variable itself.

6.2 * is dereference

```
int num = 6;

int* pointer = &num;



cout << *pointer << endl;
```

*pointer means: go to the address stored inside pointer, and take out the content there. So it outputs 6.

If you write:

```
*pointer = 100;
```

This is not changing the address stored inside pointer. It is changing the content at the address pointer points to. Since pointer points to num, num becomes 100.

7. Pointer pointing to a struct: object uses ., pointer uses ->

```
struct Item {

    int id;

    string name;

    double price;

};



Item item;

Item* p = &item;
```

item is an Item object. p is an Item pointer, and p stores the address of item. Because p is a pointer, use -> to access members:

```
p->id = 1;

p->name = "soda";

p->price = 2.99;
```

This is equivalent to:

```
(*p).id = 1;
```

```
(*p).name = "soda";

(*p).price = 2.99;
```

Special reminder: objects use ., pointers use ->. Core formula:

```
p->price == (*p).price
```

8. new/delete: dynamically request memory

Normal local variables usually live on the stack and disappear automatically when the function ends. Objects created by new live on the heap, and we must delete them ourselves.

```
int* p = new int;

*p = 88;
```

new int does two things:

1. It requests a piece of heap memory that can hold an int.

2. It returns the address of that memory.

So the left side needs int* p to receive this address.

To release it:

```
delete p;

p = nullptr;
```

delete p does not delete the pointer variable p itself. It releases the heap memory that p points to. After delete, p may still store the old address. This is called a dangling pointer. So it is better to write p = nullptr.

8.1 When can we delete?

Special reminder: only things created by new can be deleted. Things not created by new must not be deleted.

Wrong example:

```
Item item;

Item* p = &item;
```

```
delete p;  // wrong: item was not created by new
```

Correct example:

```
Item* p = new Item;



p->id = 1;

p->name = "soda";

p->price = 2.99;



delete p;

p = nullptr;
```

8.2 new/delete and new[]/delete[] must match

Creation method	Release method
new int	delete p
new Item	delete p
new int[size]	delete[] p
new Item[size]	delete[] p

If you create a group of objects, use delete[].

```
Item* items = new Item[size];



delete items;    // wrong

delete[] items;  // correct
```

9. Dynamic array: pointer stores the beginning address

```
int size;

cin >> size;



int* nums = new int[size];
```

new int[size] creates a group of int values. It returns the address of the 0th int. So nums stores the beginning address of the dynamic array.

Core relations:

```
nums[0] == *nums

nums[1] == *(nums + 1)

nums[i] == *(nums + i)
```

The key idea of pointer arithmetic is: p + 1 does not mean adding 1 byte to the address. It means moving to the next object of the same type.

Special reminder: if p is int*, then p + 1 jumps over one int. If p is Coordinate*, then p + 1 jumps over one Coordinate.

10. Dynamic struct array: the hardest part of Chapter 4

```
struct Coordinate {

    int id;

    string name;

    float Xpoint;

    float Ypoint;

    float Zpoint;

};



int size;

cin >> size;



Coordinate* coor_log = new Coordinate[size];
```

Here, coor_log is a Coordinate*. It stores the address of the 0th Coordinate in the dynamic array. We can create a second pointer to practice pointer arithmetic:

```
Coordinate* pointer = &coor_log[0];
```

This line initializes the position of pointer, making pointer point to the 0th Coordinate. Actually, the two lines below are equivalent:

```
Coordinate* pointer = &coor_log[0];

Coordinate* pointer = coor_log;
```

Because:

```
coor_log == &coor_log[0]
```

10.1 Address, object, member: do not mix the three layers

Expression	What it is	Meaning
pointer + i	Address	Address of the ith Coordinate
&coor_log[i]	Address	Address of the ith Coordinate
*(pointer + i)	Object	The ith Coordinate object itself
coor_log[i]	Object	The ith Coordinate object itself
(pointer + i)->Xpoint	Member	Xpoint member of the ith Coordinate
coor_log[i].Xpoint	Member	Xpoint member of the ith Coordinate

So remember these three formulas:

```
pointer + i == &coor_log[i]

*(pointer + i) == coor_log[i]

(pointer + i)->Xpoint == coor_log[i].Xpoint
```

Special reminder: compare address with address, object with object, and member with member. Do not compare an address with an object.

10.2 Use pointer arithmetic when inputting data

```
for (int i = 0; i < size; i++) {

    int id_coor;

    cin >> id_coor;

    (pointer + i)->id = id_coor;



    string name_coor;
```

```
    cin >> name_coor;
    (pointer + i)->name = name_coor;


    float X;
    cin >> X;
    (pointer + i)->Xpoint = X;
}
```

Here, (pointer + i) is the address of the ith Coordinate, so we use -> to access its members.

10.3 Output can use array indexing

```
for (int a = 0; a < size; a++) {
    cout << coor_log[a].id << endl;
    cout << coor_log[a].name << endl;
    cout << coor_log[a].Xpoint << endl;
}
```

Here, coor_log[a] is the ath Coordinate object, so we use . to access members.

Special reminder: coor_log[a].Xpoint and (pointer + a)->Xpoint are the same thing, just written in different styles.

10.4 What is pointer verification?

Pointer verification is not new syntax, and it is not a real program feature. It just prints two different writing styles to see whether they access the same object.

```
cout << coor_log[0].name << endl;
cout << pointer->name << endl;


cout << coor_log[1].name << endl;
cout << (pointer + 1)->name << endl;
```

If the output is the same, it proves:

```
coor_log[0].name == pointer->name
```

```
coor_log[1].name == (pointer + 1)->name
```

This uses output to prove that array indexing and pointer-offset writing access the same block of data.

11. Do not move the original pointer

This point is very important. Suppose:

```
Coordinate* coor_log = new Coordinate[size];
```

coor_log stores the original address returned by new[]. At the end, delete[] must use this original address.

Dangerous writing:

```
coor_log = coor_log + 1;

delete[] coor_log;  // wrong: coor_log is no longer the beginning of the array
```

Safe writing:

```
Coordinate* coor_log = new Coordinate[size];

Coordinate* pointer = coor_log;


(pointer + i)->Xpoint = X;


delete[] coor_log;
```

In other words: keep coor_log as the original address for delete[]. Use pointer for access and offset calculations.

12. The order of delete and nullptr

Wrong order:

```
coor_log = nullptr;

delete[] coor_log;
```

Why is this wrong? You first turn coor_log into nullptr, so you lose the original address returned by new[]. The original dynamic array can no longer be released. This causes a memory leak.

Correct order:

```
delete[] coor_log;

coor_log = nullptr;

pointer = nullptr;
```

Special reminder: delete first, then nullptr. If coor_log and pointer point to the same dynamic array, release it only once. Do not delete[] coor_log and then delete[] pointer.

13. cin input format: do not type f or commas in the terminal

If your code is:

```
float X;

cin >> X;
```

In the terminal, enter:

```
3.12
```

Do not enter:

```
3.12f
```

3.12f is a float literal written inside C++ code. It is not the normal way to type data in the terminal.

Likewise, if your code reads values continuously using cin >>:

```
cin >> id;

cin >> name;

cin >> X;

cin >> Y;

cin >> Z;
```

Your input should use spaces:

```
1 MarsBase 10.5 20.0 30.2
```

Do not use commas:

```
1, MarsBase, 10.5, 20.0, 30.2
```

14. Common bug memo

Wrong writing	Problem	Correct direction
int size; getline(cin, size);	getline cannot directly read int	cin >> size;
float X; getline(cin, X);	getline cannot directly read float	cin >> X;
Coordinate coor_log = new Coordinate[size];	new[] returns an address, not an object	Coordinate* coor_log = new Coordinate[size];
Item* p = item[0];	item[0] is an object, not an address	Item* p = &item[0]; or Item* p = item;
if ((pointer + i) == coor_log[i])	left side is address, right side is object	compare (pointer + i) with &coor_log[i]
coor_log = nullptr; delete[] coor_log;	the original address is lost before delete	delete[] first, then set nullptr
delete[] coor_log; delete[] pointer;	same memory released twice	release once, then set both pointers to nullptr

15. Final Chapter 4 summary

- An array is a row of continuous boxes.
- A C-style string is a char array ending with \0.
- string is a more convenient string object. It can use .size(), +, and +=.
- struct combines a group of related data into a new type.
- Objects access members with ., and pointers access members with ->.
- A pointer is also a variable, but it stores an address.
- &x means the address of x. *p means the content at the address p points to.
- p->member is equivalent to (*p).member.
- new creates heap memory. delete releases one object created by new.
- new[] creates a group of objects. delete[] releases a group of objects.
- After delete, it is best to set the pointer to nullptr.
- array[i] is equivalent to *(array + i).
- structArray[i].member is equivalent to (structArray + i)->member.
- delete[] must use the original beginning address of the array.

16. Self-check: if you can understand these, you can enter Chapter 5

You do not need to write a full project. First, be able to answer these questions:

1. In int* p = #, what are p, &num, *p, and &p?
2. Can you write delete p after Item item; Item* p = &item;? Why?
3. When should Item* p = new Item; be deleted?
4. For Item* items = new Item[size];, should release use delete or delete[]?
5. What are items[0], &items[0], and items?
6. Are items[i] and *(items + i) the same object?
7. Are items[i].price and (items + i)->price the same member?
8. Why should you not write items = items + 1 and then delete[] items?